

# Web Application Security Assessment Report

Target Website: **OWASP Juice Shop**

Prepared By: **Adithyan.V**

Date: **June 01, 2026**

## **1. Introduction**

- Think of a website like a digital building. To make sure it is safe, we need to check if the locks on the doors work and if there are any hidden cracks that an intruder could use to sneak inside.
- For this project, we tested the OWASP Juice Shop and we used a professional tool called OWASP ZAP to scan the site for weaknesses and safety gaps. Our goal was to find these issues and explain how to fix them in simple language.

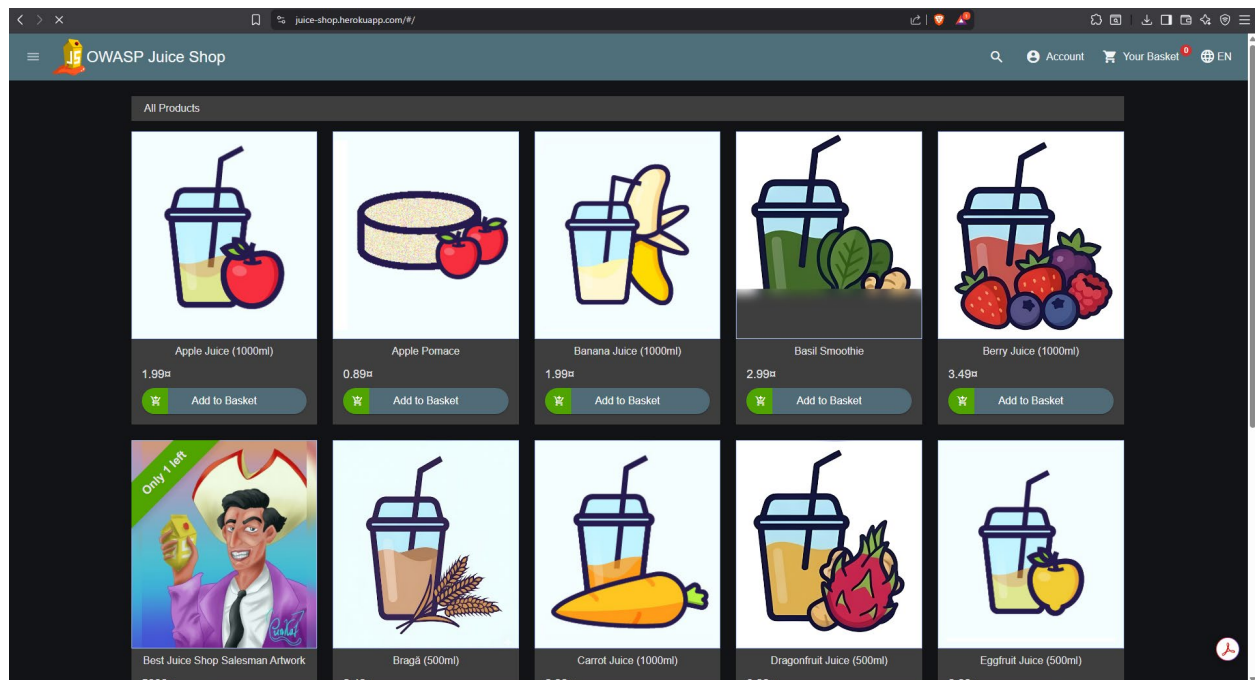
## **2. Tools & Testing Method**

- **ZAP Setup:** We set up a "proxy" so our testing tool could watch the data moving between the browser and the website.
- **Site Mapping:** We explored the website to find all its pages and buttons.
- **Scanning:** We ran an automated scan to identify common security flaws.
- **Review:** We checked the results to make sure the findings were real and important.

## **3. Scope of the Test**

Target URL: <https://juice-shop.herokuapp.com>

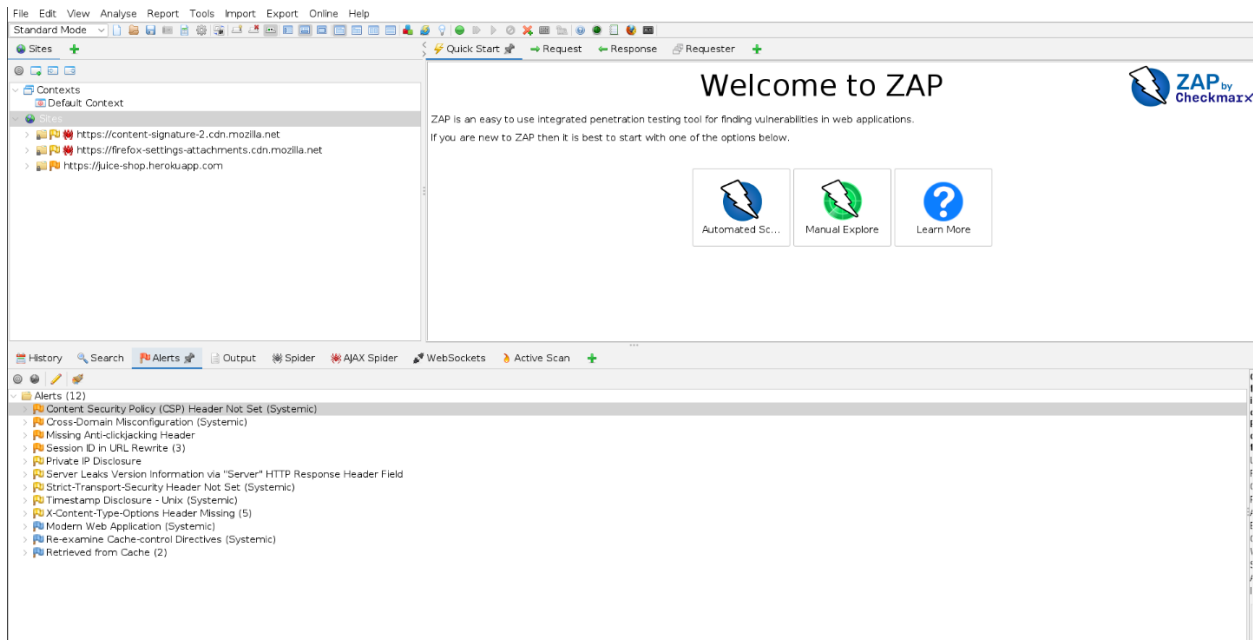
The **OWASP Juice Shop** is a widely recognized, modern web application designed specifically for security testing and educational purposes. It mimics a real-world e-commerce platform where users can browse products, add items to a shopping cart, and log into user accounts.



## 4. Summary of Findings

| Risk Level    | What It Means                                     | Count |
|---------------|---------------------------------------------------|-------|
| Medium        | Important issues that could affect user accounts. | 2     |
| Low           | Minor issues or missing "extra" safety layers.    | 7     |
| Informational | Basic notes about how the site is built.          | 3     |

## 5. List of Vulnerabilities Found



- Content Security Policy (CSP) Header Not Set (Systemic)
- Cross-Domain Misconfiguration (Systemic)
- Missing Anti-clickjacking Header
- Session ID in URL Rewrite (3)
- Private IP Disclosure
- Server Leaks Version Information via "Server" HTTP Response Header Field
- Strict-Transport-Security Header Not Set (Systemic)
- Timestamp Disclosure - Unix (Systemic)
- X-Content-Type-Options Header Missing (5)
- Modern Web Application (Systemic)
- Re-examine Cache-control Directives (Systemic)
- Retrieved from Cache (2)

## 6. Main Vulnerabilities & Easy Fixes

### Finding 01: Session ID in URL Rewrite

- What it is: The website puts private session keys in the web address bar.

- The Risk: Shared links or history could leak the key, allowing hijackers to log in without a password.
- The Fix: Stop putting keys in the URL; use secure HttpOnly cookies instead.

### **Finding 02: Cross-Domain Misconfiguration**

- What it is: Permission settings allow unrelated websites to request data.
- The Risk: Malicious background sites could steal private info while a user is logged in.
- The Fix: Use a whitelist of trusted websites for data interaction.

### **Finding 03: Missing Security Headers**

- What it is: Missing instruction tags like CSP or Anti-Clickjacking.
- The Risk: Browsers won't block malicious code or hidden click-traps.
- The Fix: Add security headers to server settings to activate browser shields.

## **7. Conclusion**

This test showed that while the website works well, it is missing some important safety settings. The most important step is to stop showing session keys in web addresses and to add the missing security "instruction tags" to the server. Doing this will make the website much safer for everyone.